FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Asmaa Khaled Elsayed Esbitan	Saudi Arabia	asmaesbitan@gmail.com	
Caleb Gichuki Mwangi	Kenya	calebgichuki12@gmail.com	
Ruthvik Mohan Menduguttala	India	ruthvikmohan007@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an "X" above).	
Team member 1	Ruthvik Mohan Menduguttala
Team member 2	Caleb Gichuki Mwangi
Team member 3	Asmaa Khaled Elsayed Esbitan

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed. **Note:** You may be required to provide proof of your outreach to non-contributing members upon request.

Table Content

Issue 1

Issue 1: Optimizing Hyperparameters

Technical Section

Definition

- Hyperparameters are **model settings chosen before training** that control how the learning process unfolds.
- Unlike parameters (weights, coefficients) that are learnaing during training, hyperparameters are external controls.

Examples of Hyperparameters in Finance Models

- **Ridge/LASSO regression:** penalty parameter λ.
- **Decision trees:** maximum depth, minimum samples per split, criterion (Gini/entropy).
- **k-means clustering:** number of clusters (k).
- **Support Vector Machines:** regularization parameter (C), kernel type, gamma.
- **Neural networks:** learning rate, number of hidden layers, activation functions, batch size, epochs.

Optimization Methods

- **Grid Search:**
  - Exhaustive search over all combinations.
  - Guarantees coverage but computationally expensive.
- **Random Search:**
  - Samples random combinations.
  - More efficient for high-dimensional hyperparameter spaces.
- **Bayesian Optimization:**
  - Uses probabilistic models to guide search.
  - Balances exploration and exploitation.
- **Cross-Validation:**
  - Splits data into folds to evaluate performance across multiple subsets.
  - Reduces risk of overfitting to a single train/test split.

Equations

- **Cross-Validation Error:**

$$CV(\theta) = \frac{1}{K}\sum_{k=1}^{K} L(y^{(k)}, f_\theta(X^{(k)}))$$

where (K) = number of folds, (L) = loss function, (θ) = hyperparameter set.

- **Regularization Example (Ridge):**

$$\min_{\beta}\left\{\sum_{i=1}^{n}(y_i - X_i\beta)^2 + \lambda\sum_{j=1}^{p}\beta_j^2\right\}$$

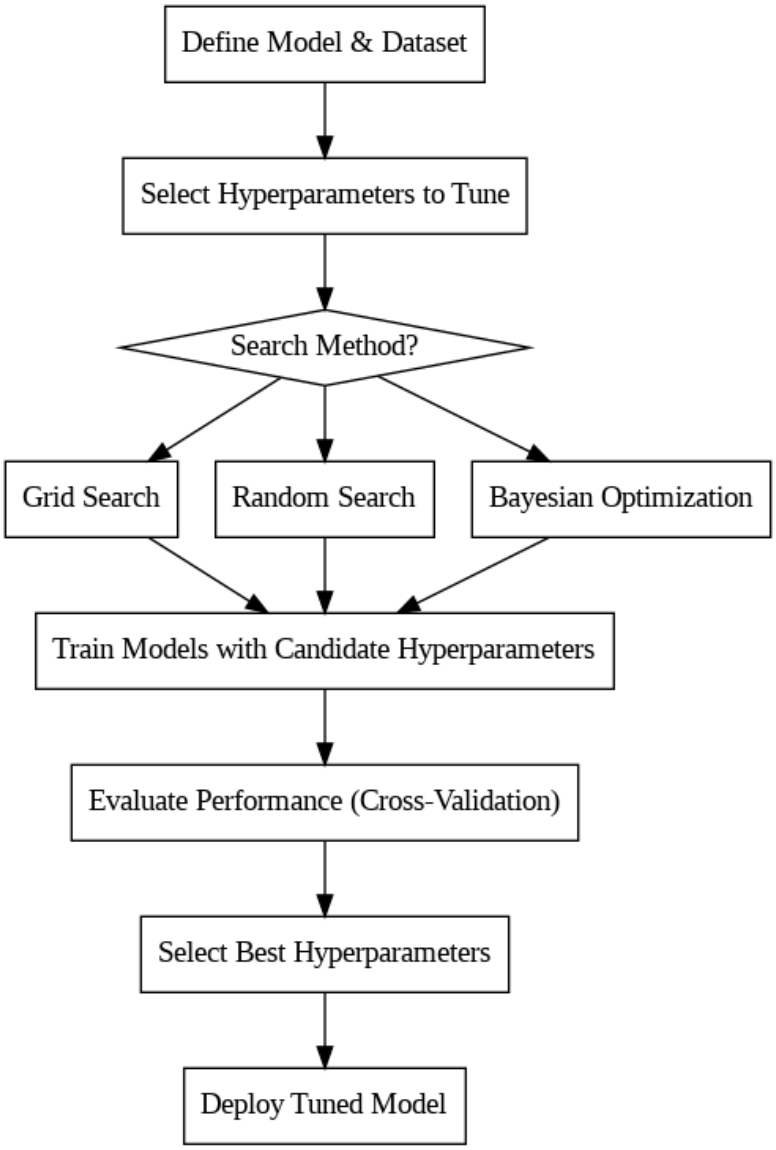
Features of Hyperparameter Tuning

- Ensures models generalize well to unseen data.
- Provides balance betwen accuracy and computational efficiency.
- Different models require different tuning strategies.

Guide (Inputs/Outputs)

- **Inputs:** dataset, model type, hyperparameter ranges.
- **Outputs:** best hyperparameter set, performance metrics (accuracy, RMSE, Sharpe ratio).

Illustration



Non-Technical Section

Why Hyperparameters Matter

- Hyperparameters are like the **settings on a machine**. If set too high or too low, the machine won't perform optimally.
- In finance, this means the difference between a model that predicts market trends accurately and one that misses opportunities.

How We Optimize Them

- We test different settings systematically.
- For example, we might try different "depths" for a decision tree until we find the one that balances accuracy and stability.
- We use methods like **cross-validation** to ensure the chosen settings work not just on past data but also on new and unseen data.

How We Know They Are Optimal

- The chosen hyperparameters minimize error across multiple test scenarios.
- They produce consistent results, not just one-off successes.
- They balance **accuracy** (fit to past data) and **robustness** (ability to generalize).

Client-Facing Explanation

- Think of hyperparameters as the **knobs and dials** on a radio.
- If tuned correctly, you get a clear signal (accurate predictions).
- If tuned poorly, you get static (noisy, unreliable predictions).
- Our process ensures the "radio" is tuned to the right frequency for stable performance.

Contribution to Group Technical Section

- Hyperparameter tuning examples from my models:
  - **Clustering:** number of clusters (k).
  - **Trees:** maximum depth, minimum samples per split.
- These examples will be integrated into the group's **Technical Section** to show how models in general require tuning.

Journal Reference

Bergstra, James, and Yoshua Bengio. "Random Search for Hyper-Parameter Optimization." *Journal of Machine Learning Research*, vol. 13, 2012, pp. 281–305.

Issue 2: Optimizing the Bias-Variance Tradeoff

Technical Section

In this issue, I'm talking about how to tell if a machine learning model will actually hold up on future financial data. It's crucial because a model shouldn't just ace the training set it need to work on fresh, unseen data too. The bias-variance tradeoff is what helps me see if a model is to simple or too complex. Bias is when the model is too basic and misses key patterns; think of a naive stock predictor that overlooks major market signals. That's underfitting. Variance is the opposite, where the model clings to every little noise in the training data. A super deep decision tree might crush it on training data but flop on test data classic overfittng.

The prediction error can be explained by this equation:

$$\text{Expected Test Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

So it's about finding the right balance flexible enough to catch actual patterns but not so flexible it memorizes the noise. To check this, I split the data into training and testing sets. The training set builds the model, and the testing set checks how it handles new, unseen data. Since this is finance, I need to split by time: train on older data, test on newer data. That avoids look-ahead bias, so the model doesn't use future info to predict the past.

The steps I'd take are:

- Arrange the financial data by date.
- Take the earlier part as the training set.
- Use the later part as the testing set.
- Compare the training and testing results.
- Pick the model that perform well on the testing data without overfitting.

If the training result is very high but the testing result is much lower, are means the model probably have high variance and is overfitting.

If both training and testing results are weak, this means the model probably have high bias and is underfitting.

If the training result and testing result are both good and close to each other, this means the model haave a better bias-variance balance.

For financial models I should not only look at accuracy. I should also look at financial performances measures such as return, Sharpe ratio, maximum drawdown and transaction costs. A model can have good accuracy but still not be profitable in real trading.

Non-Technical Section

In simple words and this issue is about checking whether my model can work well in the future.

This is important in finance because markets change. A model may look good when I test it on old data, but it may not work well in the future. For example, a model trained during a strong market may not work well during a crisis or a period of high interest rates.

The model should learn useful market signals, not just memorize past data.

If the model is too simple, it may miss important information. If the model is too complex, it may learn random movements that will not happen again. So, I need to choose a model that is balanced.

For example, if a decision tree gives very high results on training data but low results on testing data, I would not trust it. This mean it may be overfitting.

If another model gives slightly lower training results but better and more stable testing results, I would prefer that model. This is because investors care about future performance, not only past performance.

My practical recommendation is that portfolio strategists should choose the model that works best on testing data, not the model that only looks best on training data.

The best model should:

- Perform well on future-like data.

- Have a small difference betwen training and testing results.
- Be stable in different market periods.
- Consider transaction costs.
- Improve financial decision-making.

So, for Issue 2, my conclusion is that the best model is not always the most complex model. The best model is the one that gives reliable results on new financial data.

Issue 3

Issue 3: Applying Ensemble Learning: Bagging, Boosting, or Stacking

Technical Section

In this issue, I explain how different machine learning models can be used together. This is called ensemble learning.

The main idea of ensemble learning is to combine several models to make one final prediction. This can improve prediction performance because one model may make mistakes, but several models together can reduce those mistakes.

this is useful because the market is noisy and uncertain. One model may not always capture all market patterns. By combining models and I may get a more stable prediction.

There are three main ensemble methods:

Method	**Meaning**	**Main Purpose**	**Finance Example**

Bagging	Combines many similar models trained on different samples	Reduces variance and improves stability	Random Forest for predicting stock price direction
Boosting	Builds models sequentially, where each model improves on the previous one	Reduces bias and improves accuracy	Gradient Boosting for predicting default risk
Stacking	Combines different types of models using a final meta-model	Uses the strengths of different models	Combining logistic regression, random forest, and boosting for credit risk prediction

Bagging

Bagging means building many versions of the same model using different samples of the training data. Then the predictions is combines.

For classification problems and the final answer is usually based on majority voting.

$$\hat{y}_{ensemble} = \text{majority vote}$$

For regression problems and the final prediction is the average of all model predictions.

$$\hat{y}_{ensemble} = \frac{1}{M}\sum_{m=1}^{M}\hat{y}_m$$

Where:

- M is the number of models.
- $\hat{y}_m$ is the prediction from one model.

Boosting

Boosting builds models one after another. Each new model tries to fix the mistakes of the previous models.

The general form is:

$$\hat{y}_{ensemble} = \sum_{m=1}^{M} \alpha_m h_m(x)$$

where:

- $h_m(x)$ is one weak model.
- α_m is the weight of that model.
- M is the number of models.

Boosting can be very powerful because it focuses on the difficult cases. However, if boosting is too complex, it can overfit. For example if I use too many trees or a very high learning rate, the model may fit noise instead of real signals.

In finance, boosting can be used for predicting credit risk and stock return direction, volatility, or default probability.

Stacking

Stacking combines different types of models. For example, I can combine logistic regression and random forest and and gradient boosting.

Each model makes its own prediction. Then a final model combines these predictions into one final output.

The stacking equation is:

$$\hat{y}_{stack} = g(\hat{y}_1, \hat{y}_2, \hat{y}_3, ..., \hat{y}_k)$$

where:

- $\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_k$ are predictions from the first models.
- g is the final model that combines them.

Stacking is useful because different models may find different patterns. For example, logistic regression may find simple relationships, while random forest may find non-linear patterns.

Non-Technical Section

This is similar to investing in a portfolio. Investors usually do not put all their money in one stock because that is risky. In the same way and I should not always depend on only one machine learning model.

Different models may have different strengths. One model may work better in normal market conditions, while another model may work better when the market is volatile. By combining models and I may get a more balanced and reliable prediction.

For example:

- Logistic regression can explain simple relationships.
- Random forest can reduce unstable predictions.
- Boosting can improve weak predictions.
- Stacking can combine different model opinions.

This is useful for portfolio strategists because financial markets is not stable all the time. A model that works well today may not work well later. Using ensemble learning can reduce the risk of depending too much on one model.

However, I should not automatically assume that ensemble models is always better. I still need to test them on out-of-sample data. If the ensemble does not improve the testing result, then the simpler model may be better.

My recommendation is:

1. First, use the best single model as the benchmark.
2. Then test a bagging model such as Random Forest.
3. Then test a boosting model such as Gradient Boosting.
4. If different models give useful results, test stacking.
5. Choose the model that gives the best testing result and is useful for financial decisions.

For Issue 3 my conclusion is that ensemble learning can help improve financial predictions, but only if the combined model performs better on new data.